



Centrifuge (ChainlinkAdapter) Security Review

Reviewed by: Goran Vladika, SpicyMeatball, dingbats, Mirko Pezo

17th October - 21st October, 2025

Centrifuge (ChainlinkAdapter) Security Review Report

Burra Security

October 22, 2025

Introduction

A time-boxed security review of the **ChainlinkAdapter** built by the Centrifuge team was done by **Burra Security** team, focusing on the security aspects of the smart contracts.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

About Burra Security

Burra Sec offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations.

About ChainlinkAdapter

The focus on the security review was the [ChainlinkAdapter](#) contract that enables cross-chain communication between different blockchains using the Chainlink CCIP protocol.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - The technical, economic, and reputation damage from a successful attack

Likelihood - The chance that a particular vulnerability gets discovered and exploited

Severity - The overall criticality of the risk

Informational - Findings in this category are recommended changes for improving the structure, usability, and overall effectiveness of the system.

Security Assessment Summary

review commit hash - 550a1fda42a27239aaa7cf393b0e476ef14b5914

Scope

The following smart contracts were in the scope of the audit:

- src/adapters/ChainlinkAdapter.sol
- src/adapters/interfaces/IChainlinkAdapter.sol

Findings Summary

ID	Title	Severity	Status
M-1	Strict message ordering causes message processing delays	Medium	Resolved
L-1	Wiring a centrifugeld more than once with a new chainSelector does not remove the sources mapping value for the old chain selector	Low	Resolved
I-1	Unused custom error definitions in <code>IChainlinkAdapter</code>	Info	Resolved
I-2	Unused <code>CastLib</code> library import	Info	Resolved
I-3	Outdated comments on bridge adapters	Info	Resolved

Detailed Findings

[M-01] Strict message ordering causes message processing delays

Target

- ChainlinkAdapter.sol

Severity

- Impact: Low
- Likelihood: High

Description

ChainlinkAdapter facilitates bidirectional message passing between chains by leveraging CCIP. In the `_createMessage` function, the adapter constructs CCIP messages using the `EVMExtraArgsV1` struct:

```
1 extraArgs: _argsToBytes(IClient.EVMExtraArgsV1({gasLimit: gasLimit +  
    RECEIVE_COST}))
```

This struct only includes a `gasLimit` field and lacks the `allowOutOfOrderExecution` boolean used in `GenericExtraArgsV2`:

```
1  /// @param gasLimit: gas limit for the callback on the destination
    chain.
2  /// @param allowOutOfOrderExecution: if true, it indicates that the
    message can be executed in any order relative to
3  /// other messages from the same sender. This value's default varies
    by chain. On some chains, a particular value is
4  /// enforced, meaning if the expected value is not set, the message
    request will revert.
5  /// @dev Fully compatible with the previously existing EVMExtraArgsV2
    .
6  struct GenericExtraArgsV2 {
7      uint256 gasLimit;
8      bool allowOutOfOrderExecution;
9  }
```

As a result, EVMToEVMOnRamp defaults the `allowOutOfOrderExecution` to **false**:

```
1  } else if (extraArgsTag == Client.EVM_EXTRA_ARGS_V1_TAG) {
2      // EVMExtraArgsV1 originally included a second boolean (strict)
        field which has been deprecated.
3      // Clients may still include it but it will be ignored.
4      return Client.EVMExtraArgsV2({gasLimit: abi.decode(extraArgs[4:],
        (uint256)), allowOutOfOrderExecution: false});
5  }
```

Having `allowOutOfOrderExecution` set to **false** means that strict per-sender nonce ordering is enforced by CCIP. If `allowOutOfOrderExecution` was set to **true**, EVMToEVMOnRamp would simply set nonce to 0 which indicates no ordering:

```
1  Internal.EVM2EVMMessage memory newMessage = Internal.EVM2EVMMessage
    ({
2      ...
3      // Only bump nonce for messages that specify
        allowOutOfOrderExecution == false. Otherwise, we
4      // may block ordered message nonces, which is not what we want.
5      nonce: extraArgs.allowOutOfOrderExecution ? 0 : ++s_senderNonce[
        originalSender],
```

On the destination chain, EVM2EVMOffRamp parses the nonce and, if greater than 0, checks that nonce matches the expected one (ie. there is no nonce gap):

```
1  // UNTOUCHED messages MUST be executed in order always IF message.nonce
    > 0.
2  if (originalState == Internal.MessageExecutionState.UNTOUCHED) {
3      if (prevNonce + 1 != message.nonce) {
4          // We skip the message if the nonce is incorrect, since message.
            nonce > 0.
5          emit SkippedIncorrectNonce(message.nonce, message.sender);
6          continue;
```

```
7   }  
8 }
```

If there is nonce gap message will be skipped, but TX will not revert. Skipped message can only be successfully delivered to Centrifuge for processing when nonce gap is filled.

So even though Centrifuge is designed to not care about message ordering in GMP protocol context, strict ordering will still be enforced by CCIP because `allowOutOfOrderExecution` was not explicitly set to `true`. Strict ordering will cause delays in message processing. Due to asynchronous nature of cross-chain message deliveries, some messages will arrive out-of-order to the destination. Any missing message blocks all subsequent messages from processing. This is not a permanent DOS because CCIP is expected to eventually deliver every message, however temporary liveliness issues will occur periodically since every nonce gap needs to be filled before protocol can resume processing new messages.

Proof of Concept

POC runs in the Arbitrum fork and shows that messages sent by Centrifuge have strict ordering enabled (nonce > 0). Add this test case to `ChainlinkAdapter.t.sol`:

```
1   function testOutgoingMsgNonce_POCC() public {  
2       // create adapter with actual CCIP router  
3       address ARB_CCIP_ROUTER = 0  
4           x141fa059441E0ca23ce184B6A78bafD2A517DdE8;  
5       adapter = new ChainlinkAdapter(GATEWAY, ARB_CCIP_ROUTER,  
6           address(this));  
7  
8       // wire Base  
9       uint256 BASE_CHAINLINK_CHAIN_SELECTOR = 15971525489660198786;  
10      adapter.wire(CENTRIFUGE_ID, abi.encode(  
11          BASE_CHAINLINK_CHAIN_SELECTOR, makeAddr("DestinationAdapter"  
12          ))));  
13  
14      // send msg to Base  
15      vm.deal(address(GATEWAY), 0.1 ether);  
16      vm.recordLogs();  
17      vm.prank(address(GATEWAY));  
18      adapter.send{value: 0.1 ether}(CENTRIFUGE_ID, "hello", 100_000,  
19          address(this));  
20  
21      // capture the log and check that nonce is 1 and not 0  
22      Vm.Log[] memory logs = vm.getRecordedLogs();  
23      uint64 nonce = _getMsgNonceFromEvent(logs);  
24  
25      // Assert nonce is 1  
26      console2.log("Msg nonce if ordering was not enforced: 0");
```

```
22     console2.log("Actual msg nonce extracted from CCIP event:",
23         nonce);
24 }
25 function _getMsgNonceFromEvent(Vm.Log[] memory logs) internal pure
26     returns (uint64 nonce) {
27     bytes32 eventSig = keccak256(
28         "CCIPSendRequested((uint64,address,address,uint64,uint256,
29         bool,uint64,address,uint256,bytes,(address,uint256)[]),
30         bytes[],bytes32))"
31     );
32     for (uint256 i = 0; i < logs.length; i++) {
33         if (logs[i].topics[0] == eventSig) {
34             bytes memory data = logs[i].data;
35
36             // The nonce is at position 0xE0 (224) from start of
37             // data
38             // Add 32 for the length prefix of the bytes array
39             assembly {
40                 let nonceWord := mload(add(data, 256)) // 32 + 224
41                 = 256
42                 nonce := nonceWord // Auto-truncates to uint64
43             }
44             break;
45         }
46     }
47 }
```

Run it:

```
1 forge test --mt testOutgoingMsgNonce_POC -vv --fork-url $ARB
2
3 Ran 1 test for test/adapters/unit/ChainlinkAdapter.t.sol:
4   ChainlinkAdapterTest
5 [PASS] testOutgoingMsgNonce_POC() (gas: 226832)
6 Logs:
7   Msg nonce if ordering was not enforced: 0
8   Actual msg nonce extracted from CCIP event: 1
9 Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.28s
   (7.64s CPU time)
```

Recommendation

Use `GenericExtraArgsV2` and explicitly set `allowOutOfOrderExecution` to `true` to avoid any delays related to strict message ordering.

Client

Fixed in <https://github.com/centrifuge/protocol-internal/pull/12>

BurraSec

Fix verified. Strict ordering is not enabled anymore.

[L-01] Wiring a centrifugeld more than once with a new chainSelector does not remove the sources mapping value for the old chain selector**Target**

- ChainlinkAdapter.sol

Severity

- Impact: Low
- Likelihood: Low

Description

If wire is done for the same centrifugeld more than once, it will overwrite the existing destinations[centrifugeld], but the old sources[chainSelector] will still be there, and not deleted.

Example:

1. wire(centrifugeldA, (chainSelectorX, adapterX))

sources[chainSelectorX] = A, adapterX destinations[A] = X, adapterX

2. wire(centrifugeldA, (chainSelectorY, adapterY))

sources[chainSelectorY] = A, adapterY destinations[A] = Y, adapterY

However, sources[chainSelectorX] = A, adapterX is still there, and not deleted.

Note that the other adapters follow the same pattern and have the same issue.

Recommendation

Consider to add an unwire function that takes in centrifugeId to clear the mapping first.

```
1 function unwire(uint16 centrifugeId) external auth {
2     uint64 chainSelector = destinations[centrifugeId].chainSelector;
3     if (chainSelector == 0) revert();
4     delete sources[chainSelector];
5     delete destinations[centrifugeId];
6
7     emit Unwired(centrifugeId, chainSelector);
8 }
```

and then add some input checks in wire

```
1 function wire(uint16 centrifugeId, bytes memory data) external auth
2 {
3     if (centrifugeId == 0) revert(); // new check
4     (uint64 chainSelector, address adapter) = abi.decode(data, (
5         uint64, address));
6     if (chainSelector == 0) revert(); // new check
7     if (sources[chainSelector].centrifugeId != 0) revert(); // new
8         check
9     if (destinations[centrifugeId].chainSelector != 0) revert(); //
10        new check
11     sources[chainSelector] = ChainlinkSource(centrifugeId, adapter)
12     ;
13     destinations[centrifugeId] = ChainlinkDestination(chainSelector
14         , adapter);
15     emit Wire(centrifugeId, chainSelector, adapter);
16 }
```

Alternative if there is no code change done, a comment about setting the existing chain selector for the centrifugeId to a zero address adapter before setting a new chain selector and adapter would prevent this issue as well.

Client

The alternative solution of adding comments to set an existing chain selector for the centrifugeId to a zero address adapter before setting a new chain selector and adapter has been done.

Fixed in <https://github.com/centrifuge/protocol-internal/pull/12>

BurraSec

Fix verified.

[I-01] Unused custom error definitions in IChainlinkAdapter

Target

- IChainlinkAdapter.sol#L54-L56
- IChainlinkAdapter.sol#L111-L112

Severity

Severity: INFO

Description

The `IChainlinkAdapter` file defines multiple custom errors that are not used anywhere in the codebase.

Recommendation

It is recommended to remove the definitions of these unused errors from the `IChainlinkAdapter` file.

Client

Fixed in <https://github.com/centrifuge/protocol-internal/pull/12>

BurraSec

Fix verified.

[I-02] Unused CastLib library import

Target

- `ChainlinkAdapter.sol#L26`

Severity

INFO

Description

The `CastLib` library is imported in `ChainlinkAdapter`, but it is not used anywhere in the contract.

Recommendation

Remove the `using CastLib for *;` statement and the corresponding import statement.

Client

Fixed in <https://github.com/centrifuge/protocol-internal/pull/12>

BurraSec

Fix verified.

[I-03] Outdated comments on bridge adapters**Target**

- `IAdapterWiring.sol#L6`

Severity

INFO

Description

The comments in `IAdapterWiring.sol` and `src/README.md` are outdated because they do not list the Chainlink CCIP adapter.

Recommendation

Update the comments in both files to include “Chainlink CCIP” in the list of supported adapters.

Client

Fixed in <https://github.com/centrifuge/protocol-internal/pull/12>

BurraSec

Fix verified.